

✓ Animals Data Set

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
import plotly.express as px
import plotly.graph_objects as go
```

```
df = pd.read_csv('/content/animals.csv')
display(df.head())
```

	Unnamed: 0	Animal Name	Kingdom	Phylum	Subphylum	Class	Subclass	Infraclass	Order
0	0	Arctic Wolf	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	Carnivora
1	1	Arctic Fox	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	Carnivora
2	2	Pink Fairy Armadillo	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	Cingulata
3	3	Sumatran Elephant	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	Proboscidea
4	4	Ermine	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	Carnivora

```
display(df.tail())
```

	Unnamed: 0	Animal Name	Kingdom	Phylum	Subphylum	Class	Subclass	Infraclass	Order
1344	1344	Southern Ground Hornbil	Animalia	Chordata		NaN	Aves	NaN	Bucerotiformes
1345	1345	Scarlet Ibis	Animalia	Chordata		NaN	Aves	Neornithes	Pelecaniformes
1346	1346	Rufous Hummingbird	Animalia	Chordata		NaN	Aves	NaN	Apodiformes
1347	1347	Crested Ibis	Animalia	Chordata		NaN	Aves	Neornithes	Pelecaniformes
1348	1348	Northern Goshawk	Animalia	Chordata		NaN	Aves	NaN	Accipitriformes

```
display(df.shape)
```

```
(1349, 19)
```

```
display(df.columns.tolist())
```

```
['Unnamed: 0',  
 'Animal Name',  
 'Kingdom',  
 'Phylum',  
 'Subphylum',  
 'Class',  
 'Subclass',  
 'Infraclass',  
 'Order',  
 'Superorder',  
 'Suborder',  
 'Infraorder',  
 'Family',  
 'Superfamily',  
 'Subfamily',  
 'Genus',  
 'Subgenus',  
 'Tribe',  
 'Clade']
```

```
display(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 1349 entries, 0 to 1348  
Data columns (total 19 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   Unnamed: 0      1349 non-null  int64  
1   Animal Name     1349 non-null  object  
2   Kingdom         1349 non-null  object  
3   Phylum         1349 non-null  object  
4   Subphylum     1021 non-null  object  
5   Class           1349 non-null  object  
6   Subclass        62 non-null   object  
7   Infraclass      133 non-null  object  
8   Order           1349 non-null  object  
9   Superorder      63 non-null   object  
10  Suborder        477 non-null  object  
11  Infraorder      137 non-null  object  
12  Family          1349 non-null  object  
13  Superfamily     48 non-null   object  
14  Subfamily       267 non-null  object  
15  Genus           1345 non-null  object  
16  Subgenus        18 non-null   object  
17  Tribe           49 non-null   object  
18  Clade           35 non-null   object  
dtypes: int64(1), object(18)  
memory usage: 200.4+ KB  
None
```

```
df.describe(include="all")
```

	Unnamed: 0	Animal Name	Kingdom	Phylum	Subphylum	Class	Subclass	Infraclass	
count	1349.000000	1349	1349	1349	1021	1349	62	133	
unique	NaN	1226	1	1	1	3	1	3	
top	NaN	Blue-and-Gold Macaw	Animalia	Chordata	Vertebrata	Mammalia	Neornithes	Marsupialia	Can
freq	NaN	2	1349	1349	1021	815	62	67	
mean	674.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
std	389.567067	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
min	0.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
25%	337.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
50%	674.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
75%	1011.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
max	1348.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

```
display(df.isnull().sum())
```

	0
Unnamed: 0	0
Animal Name	0
Kingdom	0
Phylum	0
Subphylum	328
Class	0
Subclass	1287
Infraclass	1216
Order	0
Superorder	1286
Suborder	872
Infraorder	1212
Family	0
Superfamily	1301
Subfamily	1082
Genus	4
Subgenus	1331
Tribe	1300
Clade	1314

dtype: int64

```
display(df.duplicated().sum())
```

```
np.int64(0)
```

```
df = df.drop_duplicates()  
display(df)
```

	Unnamed: 0	Animal Name	Kingdom	Phylum	Subphylum	Class	Subclass	Infraclass	
0	0	Arctic Wolf	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	
1	1	Arctic Fox	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	
2	2	Pink Fairy Armadillo	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	
3	3	Sumatran Elephant	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	
4	4	Ermine	Animalia	Chordata	Vertebrata	Mammalia	NaN	NaN	
...	...	...	...	...	...	...	...	...	
1344	1344	Southern Ground Hornbil	Animalia	Chordata	NaN	Aves	NaN	NaN	B
1345	1345	Scarlet Ibis	Animalia	Chordata	NaN	Aves	Neornithes	Neognathae	P
1346	1346	Rufous Hummingbird	Animalia	Chordata	NaN	Aves	NaN	NaN	
1347	1347	Crested Ibis	Animalia	Chordata	NaN	Aves	Neornithes	Neognathae	P
1348	1348	Northern Goshawk	Animalia	Chordata	NaN	Aves	NaN	NaN	A

1349 rows × 19 columns

```
display(df.shape)

(1349, 19)
```

Start coding or [generate](#) with AI.

▼ **Exploratory Data Analysis (EDA)**

```
df["Class"].value_counts()

count
Class
Mammalia    815
Aves        328
Reptilia    206

dtype: int64
```

```
df["Family"].value_counts().head(10)
```

	count
Family	
Bovidae	69
Sciuridae	59
Cercopithecidae	53
Canidae	43
Felidae	36
Mustelidae	35
Accipitridae	35
Anatidae	34
Testudinidae	29
Cervidae	28

dtype: int64

```
df["Animal Name"].nunique()
```

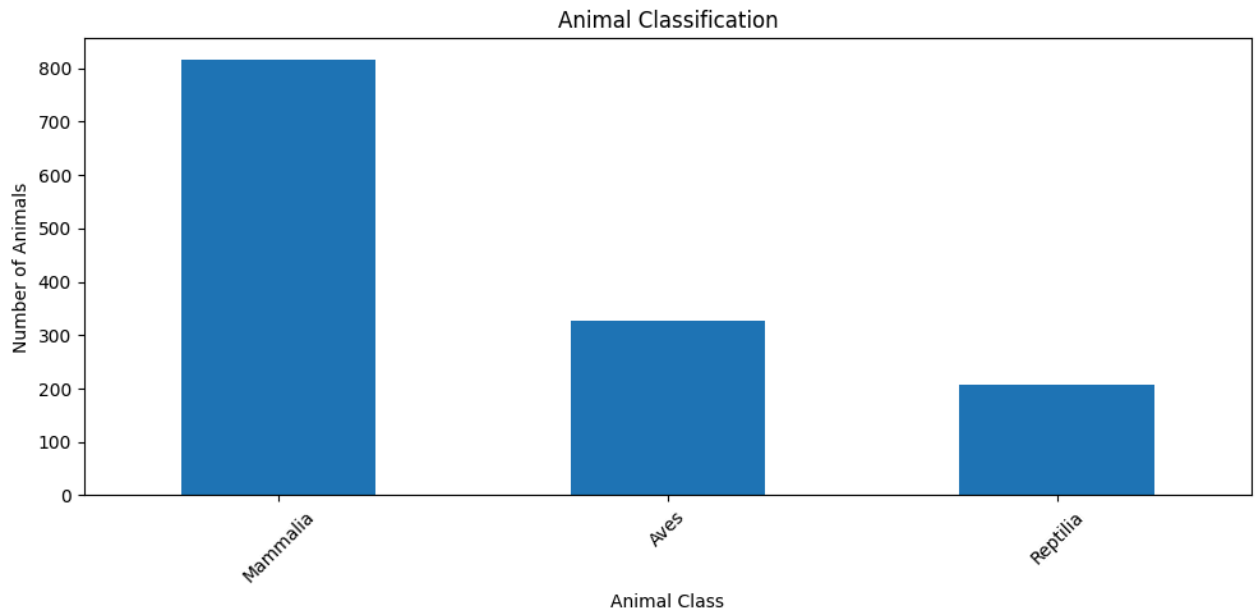
1226

Start coding or [generate](#) with AI.

✓ Visualization

```
df["Class"].value_counts().plot(
    kind="bar",
    figsize=(10, 5)
)

plt.title("Animal Classification")
plt.xlabel("Animal Class")
plt.ylabel("Number of Animals")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

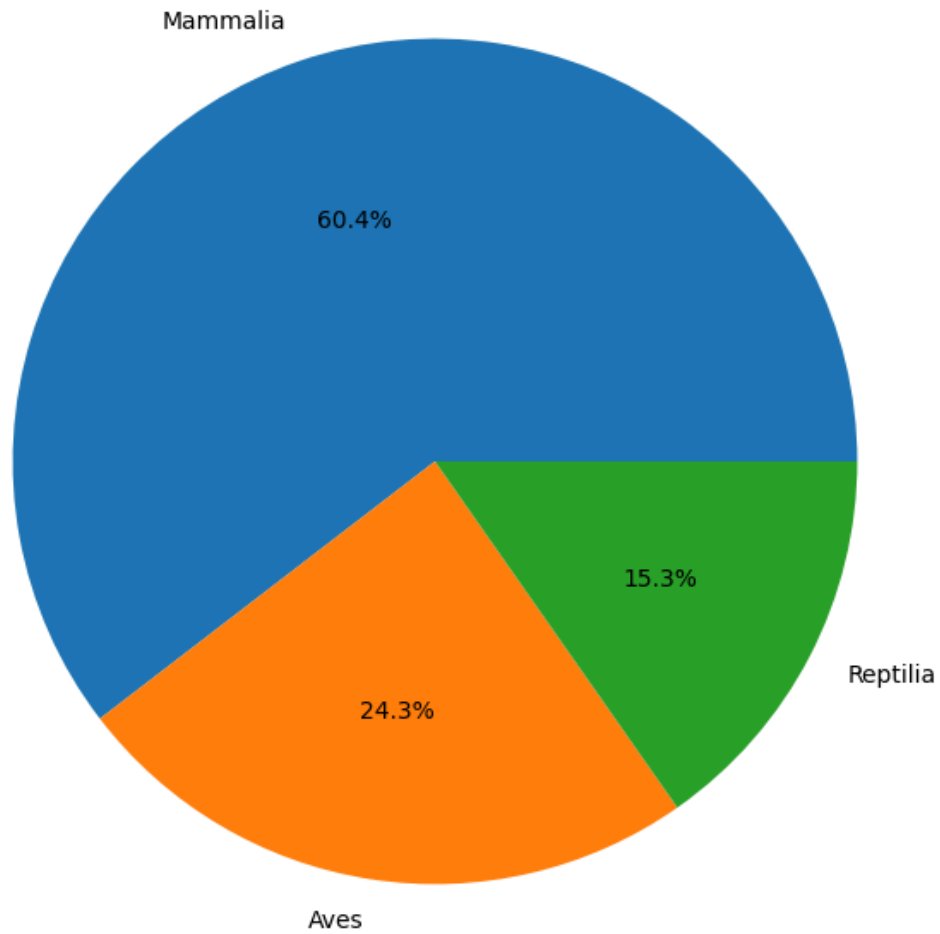


✓ **A pie chart showing the distribution of animal classes**

```
df["Class"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    figsize=(8, 8)
)

plt.title("Animal Class Distribution")
plt.ylabel("")
plt.show()
```

Animal Class Distribution



✓ Generate Business-Style Insights

```
display("Total Records:", len(df))  
display("Unique Classes:", df["Class"].nunique())  
display("Unique Families:", df["Family"].nunique())  
display("Unique Orders:", df["Order"].nunique())  
display("Missing Values:", df.isnull().sum().sum())
```



```
'Total Records:'  
1349  
'Unique Classes:'  
3  
'Unique Families:'  
210  
'Unique Orders:'  
55  
'Missing Values:'  
np.int64(12533)
```

✓ Statistical Analysis

1. Frequency Analysis Count how many animals belong to each class.

```
class_counts = df["Class"].value_counts()  
  
display(class_counts)
```

	count
Class	
Mammalia	815
Aves	328
Reptilia	206

dtype: int64

2. Percentage Analysis Calculate the percentage of animals in each class.

```
class_percentage = (  
    df["Class"].value_counts(normalize=True) * 100  
)  
  
display(class_percentage.round(2))
```

	proportion
Class	
Mammalia	60.42
Aves	24.31
Reptilia	15.27

dtype: float64

3. Analyze Animal Families Find the 10 most common animal families.

```
top_families = df["Family"].value_counts().head(10)
```

```
display(top_families)
```

Family	count
Bovidae	69
Sciuridae	59
Cercopithecidae	53
Canidae	43
Felidae	36
Mustelidae	35
Accipitridae	35
Anatidae	34
Testudinidae	29
Cervidae	28

dtype: int64

4. Cross-Tabulation Analysis Analyze the relationship between animal classes and orders.

```
class_order = pd.crosstab(  
    df["Class"],  
    df["Order"]  
)
```

```
display(class_order)
```

Order	Accipitriformes	Afrosoricida	Anseriformes	Apodiformes	Artiodactyla	Bucerot
Class						
Aves	39	0	34	7	0	
Mammalia	0	4	0	0	178	
Reptilia	0	0	0	0	0	

3 rows × 55 columns

5. Correlation Analysis First, select numerical columns.

```
numeric_df = df.select_dtypes(include="number")
```

```
correlation = numeric_df.corr()
```

```
display(correlation)
```

Unnamed: 0

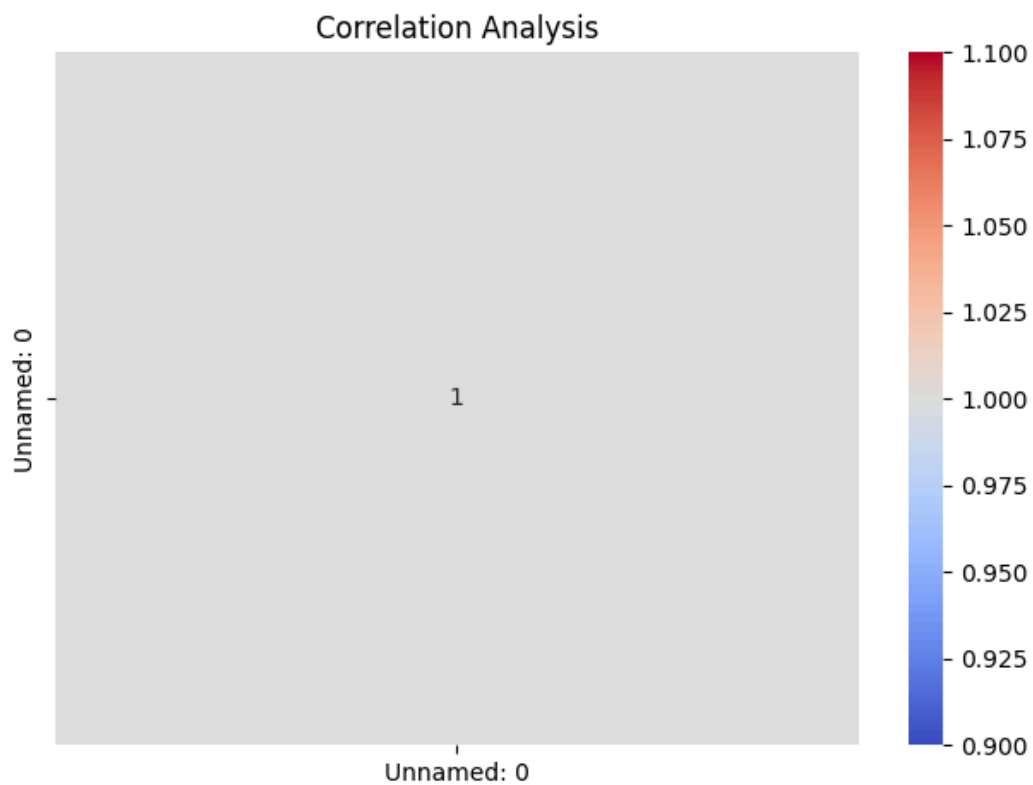
Unnamed: 0

1.0

Visualize the correlations.

```
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Analysis")
plt.tight_layout()
plt.show()
```



Step 6: Generate Business-Style Insights Now, let's create a summary of your analysis.

```
display("Total Records:", len(df))

display("Total Classes:", df["Class"].nunique())

display("Total Orders:", df["Order"].nunique())

display("Total Families:", df["Family"].nunique())

display("Most Common Class:")
display(df["Class"].value_counts().idxmax())
```

```
display("Most Common Family:")
display(df["Family"].value_counts().idxmax())
```

```
'Total Records:'
1349
'Total Classes:'
3
'Total Orders:'
55
'Total Families:'
210
'Most Common Class:'
'Mammalia'
'Most Common Family:'
'Bovidae'
```

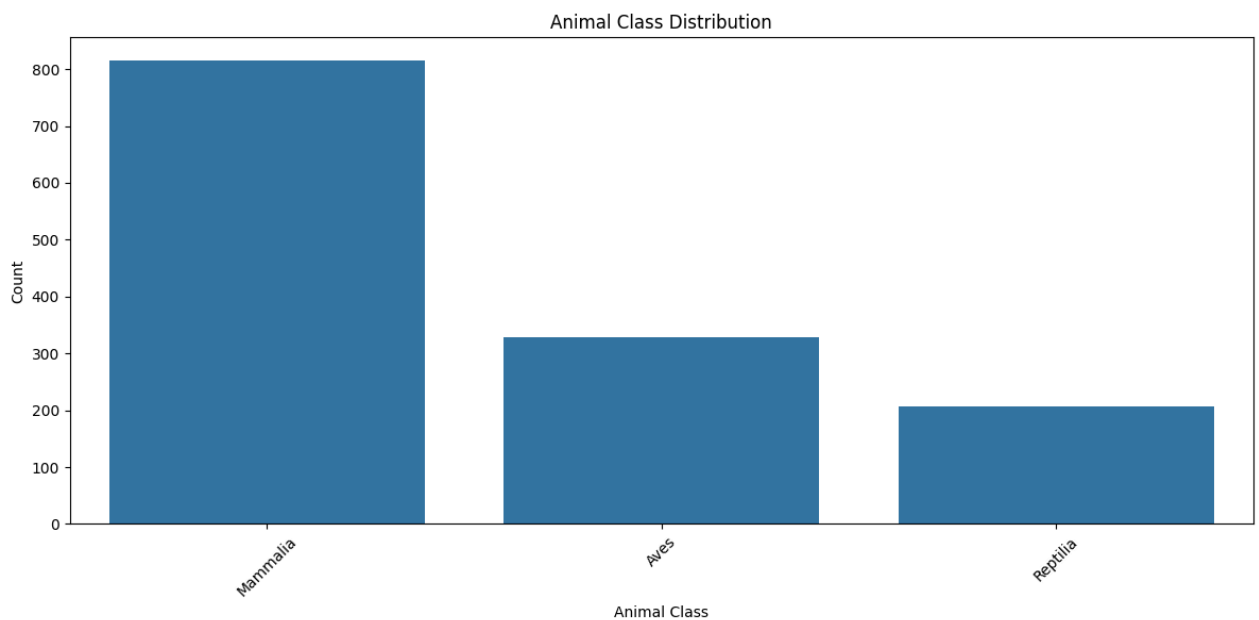
✓ Data Visualization & Key Insights

1. Animal Class Distribution

```
plt.figure(figsize=(12, 6))

sns.countplot(data=df, x="Class",
              order=df["Class"].value_counts().index)

plt.title("Animal Class Distribution")
plt.xlabel("Animal Class")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



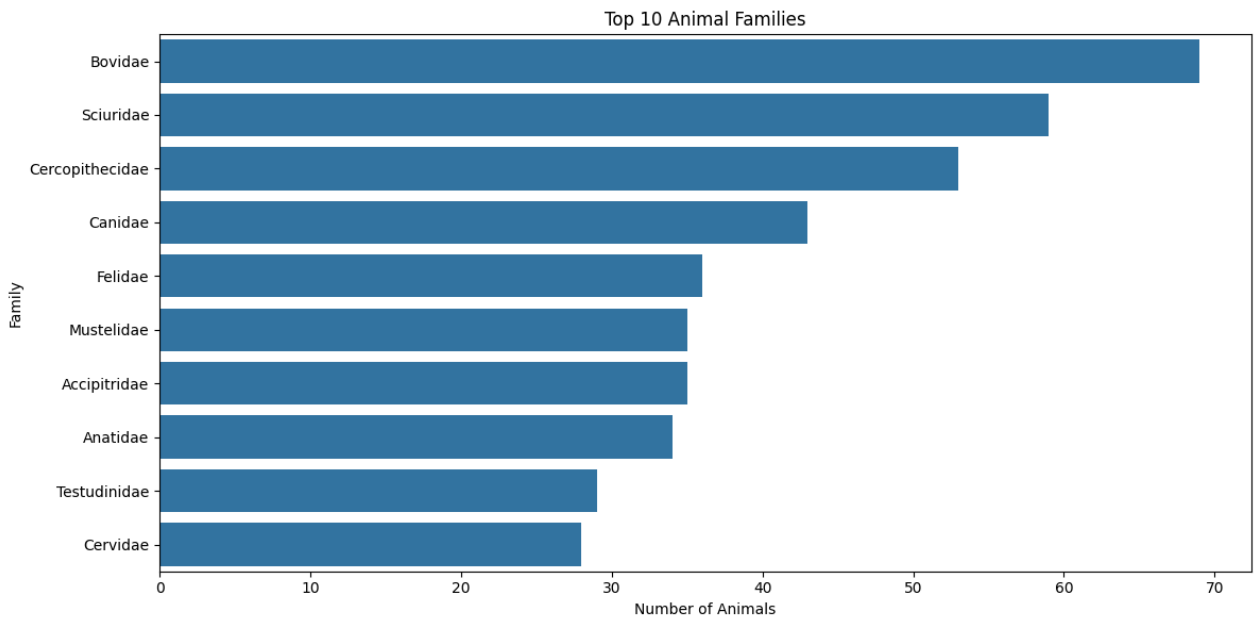
2. Top 10 Animal Families

```
plt.figure(figsize=(12, 6))

top_families = df["Family"].value_counts().head(10)

sns.barplot(x=top_families.values,
            y=top_families.index)

plt.title("Top 10 Animal Families")
plt.xlabel("Number of Animals")
plt.ylabel("Family")
plt.tight_layout()
plt.show()
```



3. Animal Class Percentage

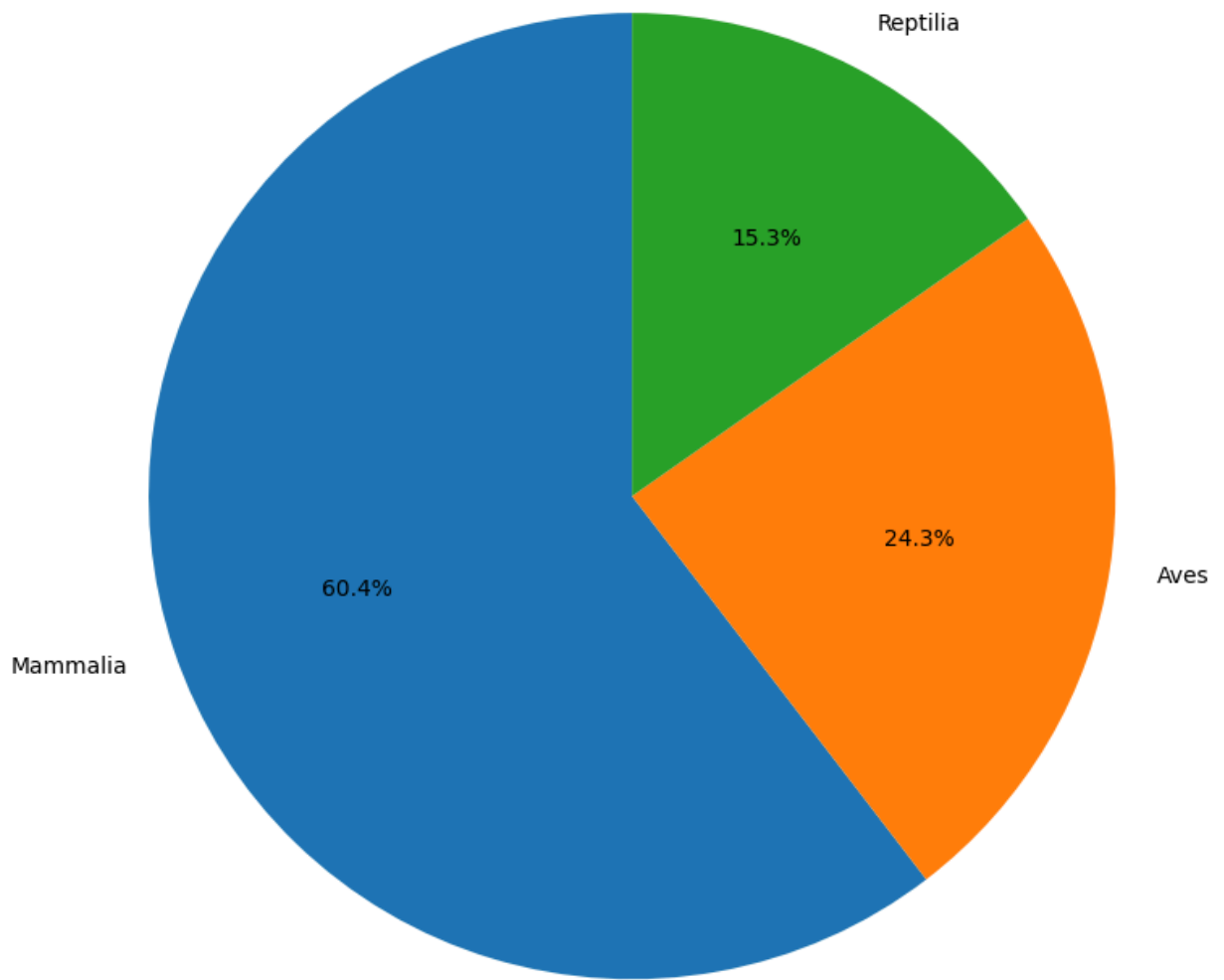
```
class_percentage = (
    df["Class"].value_counts(normalize=True) * 100
)

plt.figure(figsize=(8, 8))

plt.pie(class_percentage,
        labels=class_percentage.index,
        autopct="%1.1f%%",
        startangle=90)

plt.title("Animal Class Percentage")
plt.tight_layout()
plt.show()
```

Animal Class Percentage



4. Create a Summary of Key Insights

```

display("PROJECT INSIGHTS")

display("Total Animals:", len(df))

display("Total Classes:", df["Class"].nunique())

display("Total Families:", df["Family"].nunique())

display("Most Common Class:",
        df["Class"].value_counts().idxmax())

display("Most Common Family:",
        df["Family"].value_counts().idxmax())

```

```

'PROJECT INSIGHTS'
'Total Animals:'
1349
'Total Classes:'
3
'Total Families:'
210
'Most Common Class:'
'Mammalia'
'Most Common Family:'
'Bovidae'

```

✓ Export Your Analysis Results

Save the cleaned dataset:

```
df.to_csv("cleaned_animals.csv", index=False)
```

Export animal class counts:

```

class_summary = df["Class"].value_counts().reset_index()

class_summary.columns = ["Class", "Animal_Count"]

class_summary.to_csv("class_summary.csv", index=False)

```

Export the top 10 families:

```

family_summary = (
    df["Family"].value_counts().head(10).reset_index()
)

family_summary.columns = ["Family", "Animal_Count"]

family_summary.to_csv("top_10_families.csv", index=False)

```

Project Conclusion :

Once you've reviewed your actual results, write a conclusion covering:

- *The total number of animal records analyzed.*
- *The distribution of animals across different classes.*
- *The most frequently occurring animal families.*
- *Any important patterns discovered during EDA.*

Avoid making claims about trends or relationships that your dataset cannot establish.

✓ Create a Professional Project Report

1. Project Overview

Project Title: *Animal Classification Data Analysis Using Python*

Objective: *Analyze animal classification data using Python to identify patterns, explore animal diversity, and visualize the distribution of different animal classes and families.*

2. Create a Project Summary :

```
project_summary = {
    "Total Records": len(df),
    "Total Columns": len(df.columns),
    "Unique Classes": df["Class"].nunique(),
    "Unique Orders": df["Order"].nunique(),
    "Unique Families": df["Family"].nunique(),
    "Missing Values": df.isnull().sum().sum(),
    "Duplicate Records": df.duplicated().sum()
}

summary_df = pd.DataFrame(
    list(project_summary.items()),
    columns=["Metric", "Value"]
)

display(summary_df)
```

	Metric	Value
0	Total Records	1349
1	Total Columns	19
2	Unique Classes	3
3	Unique Orders	55
4	Unique Families	210
5	Missing Values	12533
6	Duplicate Records	0

A summary of your dataset.

3. Export Your Project Summary

```
summary_df.to_csv(
    "animal_project_summary.csv",
    index=False
)
```

4. Save Your Visualizations

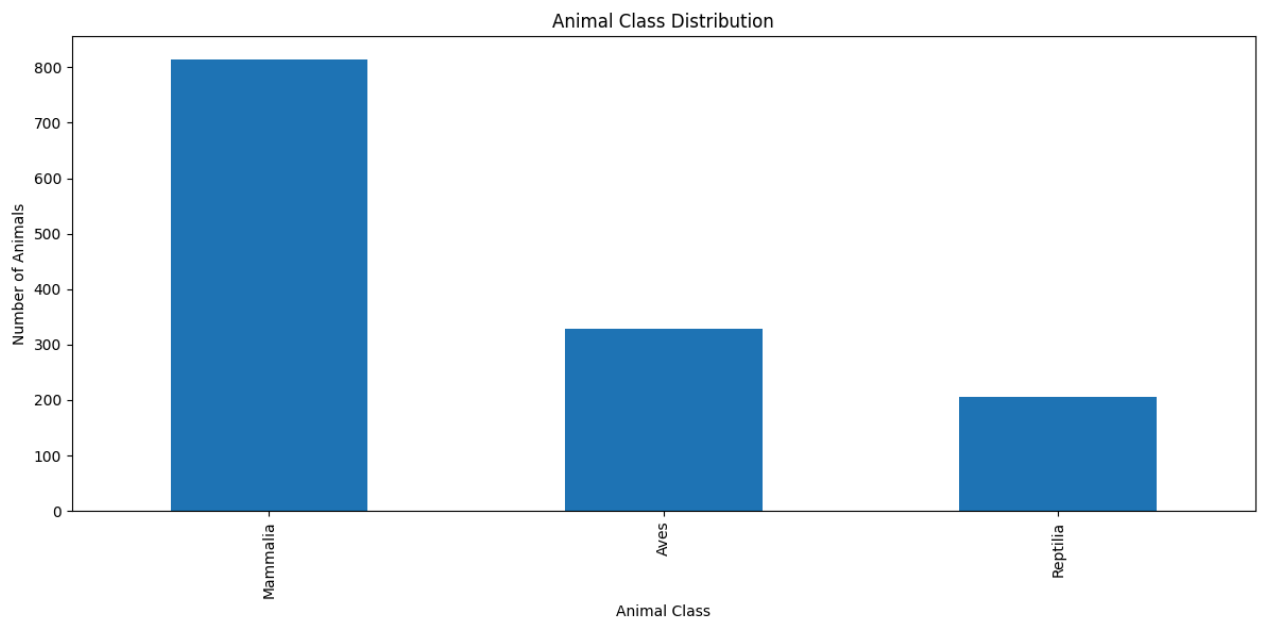
```
plt.figure(figsize=(12, 6))

df["Class"].value_counts().plot(kind="bar")

plt.title("Animal Class Distribution")
plt.xlabel("Animal Class")
plt.ylabel("Number of Animals")
plt.tight_layout()

plt.savefig(
    "animal_class_distribution.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



Feature Engineering

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
```

1. Select the target variable

```
display(df.columns.tolist())
```

```
['Unnamed: 0',  
 'Animal Name',  
 'Kingdom',  
 'Phylum',  
 'Subphylum',  
 'Class',  
 'Subclass',  
 'Infraclass',  
 'Order',  
 'Superorder',  
 'Suborder',  
 'Infraorder',  
 'Family',  
 'Superfamily',  
 'Subfamily',  
 'Genus',  
 'Subgenus',  
 'Tribe',  
 'Clade']
```

2. Prepare the dataset

```
target = "Class"  
  
# Remove rows with missing target values  
ml_df = df.dropna(subset=[target]).copy()  
  
# Select categorical features  
X = ml_df.drop(columns=[target])  
y = ml_df[target]
```

3. Encode categorical variables

```
X = X.select_dtypes(include="object")  
  
X = pd.get_dummies(X, dummy_na=True)  
  
le = LabelEncoder()  
y = le.fit_transform(y)
```

4. Split the dataset

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

Train Machine Learning Model

Decision Tree classifier.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Evaluate the Model

Calculate the model's accuracy.

```
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
```

Model Accuracy: 1.0

```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred,
        target_names=le.classes_
    )
)
```

	precision	recall	f1-score	support
Aves	1.00	1.00	1.00	66
Mammalia	1.00	1.00	1.00	163
Reptilia	1.00	1.00	1.00	41
accuracy			1.00	270
macro avg	1.00	1.00	1.00	270
weighted avg	1.00	1.00	1.00	270

1. Import the Random Forest algorithm

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
```

2. Create the model

```
rf_model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)
```

- *n_estimators=100: Creates 100 decision trees.*
- *random_state=42: Makes the experiment reproducible.*

3. Train the model

```
rf_model.fit(X_train, y_train)
```

```
▼ RandomForestClassifier ⓘ ?  
RandomForestClassifier(random_state=42)
```

4. Make predictions

```
y_pred_rf = rf_model.predict(X_test)
```

5. Evaluate model accuracy

```
accuracy_rf = accuracy_score(y_test, y_pred_rf)  
  
display("Random Forest Accuracy:", accuracy_rf)
```

```
'Random Forest Accuracy:'  
1.0
```

6. Generate a classification report

```
print(  
    classification_report(  
        y_test,  
        y_pred_rf,  
        target_names=le.classes_,  
        zero_division=0  
    )  
)
```

	precision	recall	f1-score	support
Aves	1.00	1.00	1.00	66
Mammalia	1.00	1.00	1.00	163
Reptilia	1.00	1.00	1.00	41
accuracy			1.00	270
macro avg	1.00	1.00	1.00	270

weighted avg	1.00	1.00	1.00	270
--------------	------	------	------	-----

This report includes:

- **Precision:** *How many predicted examples were correct.*
- **Recall:** *How many actual examples the model identified.*
- **F1-score:** *A balance between precision and recall.*

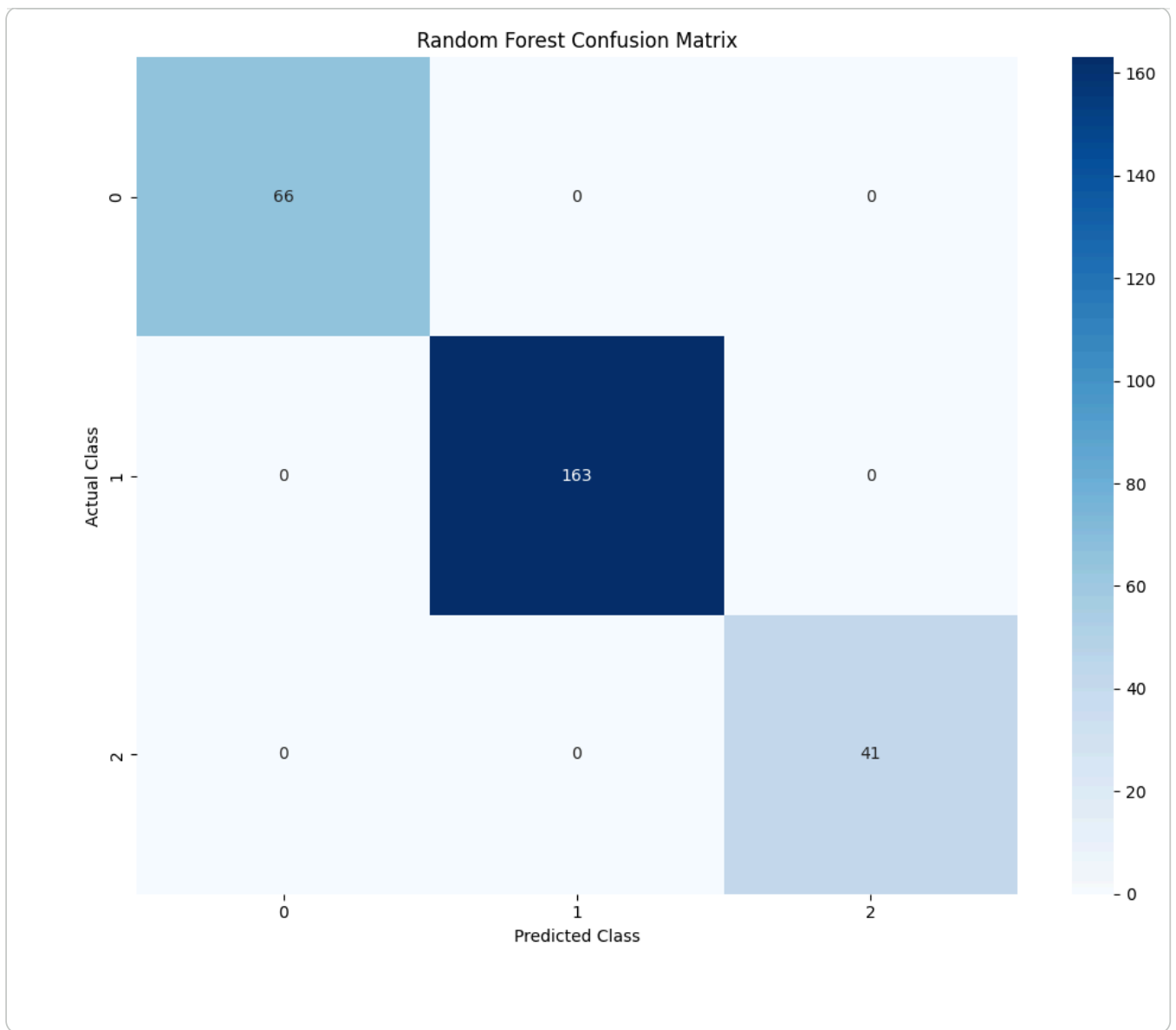
7. Create a confusion matrix

```
cm = confusion_matrix(y_test, y_pred_rf)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.tight_layout()
plt.show()
```



Compare Decision Tree and Random Forest

```
display("Decision Tree:", accuracy_score(y_test, y_pred))  
display("Random Forest:", accuracy_rf)
```

```
'Decision Tree:'  
1.0  
'Random Forest:'  
1.0
```

✓ Compare Machine Learning Models

1. Import the models

```
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.linear_model import LogisticRegression  
  
from sklearn.metrics import (
```

```
        accuracy_score,  
        precision_score,  
        recall_score,  
        f1_score  
    )
```

2. Train the models

```
models = {  
    "Decision Tree": DecisionTreeClassifier(  
        random_state=42  
    ),  
    "Random Forest": RandomForestClassifier(  
        n_estimators=100,  
        random_state=42  
    ),  
    "Logistic Regression": LogisticRegression(  
        max_iter=1000  
    )  
}
```

Train and evaluate each model:

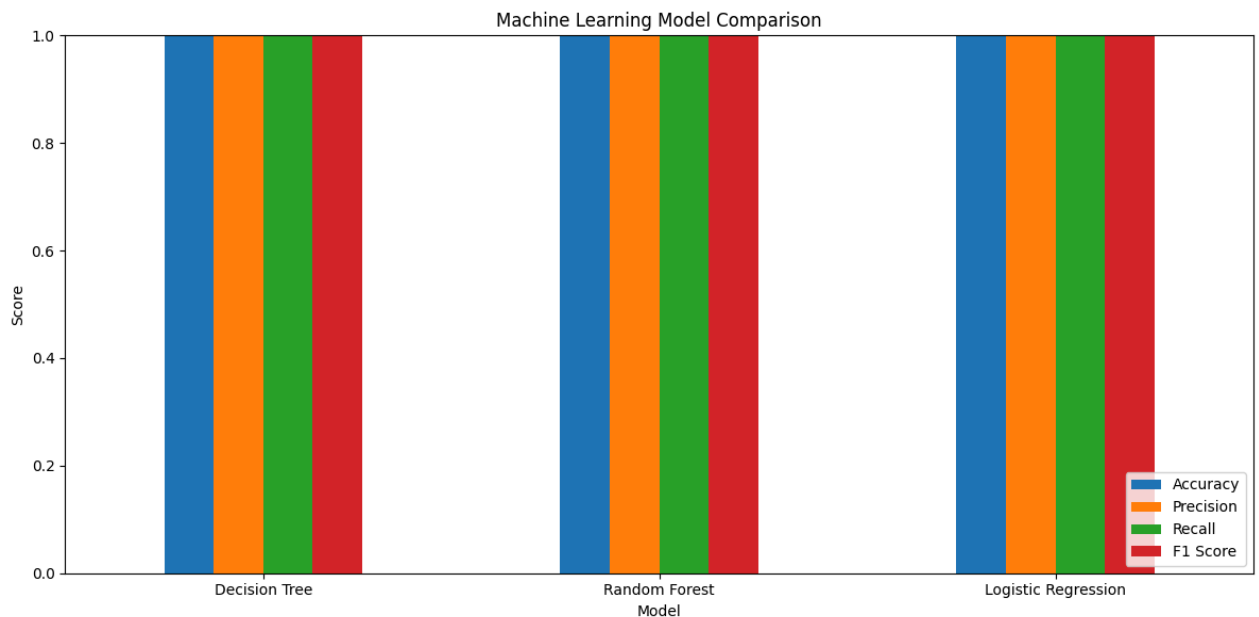
```
results = []  
  
for name, model in models.items():  
    model.fit(X_train, y_train)  
  
    y_pred = model.predict(X_test)  
  
    results.append({  
        "Model": name,  
        "Accuracy": accuracy_score(y_test, y_pred),  
        "Precision": precision_score(  
            y_test, y_pred,  
            average="weighted",  
            zero_division=0  
        ),  
        "Recall": recall_score(  
            y_test, y_pred,  
            average="weighted",  
            zero_division=0  
        ),  
        "F1 Score": f1_score(  
            y_test, y_pred,  
            average="weighted",  
            zero_division=0  
        )  
    })  
  
results_df = pd.DataFrame(results)  
  
display(results_df.round(3))
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	1.0	1.0	1.0	1.0
1	Random Forest	1.0	1.0	1.0	1.0
2	Logistic Regression	1.0	1.0	1.0	1.0

3. Visualize model performance

```
results_df.set_index("Model")["Accuracy", "Precision", "Recall", "F1 Score"]
].plot(kind="bar", figsize=(12, 6))

plt.title("Machine Learning Model Comparison")
plt.ylabel("Score")
plt.xlabel("Model")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```



4. Save the comparison results

```
results_df.to_csv(
    "model_comparison.csv",
    index=False
)
```

✓ Cross-Validation


```

from sklearn.model_selection import cross_val_score

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

scores = cross_val_score(
    rf_model,
    X_train,
    y_train,
    cv=5,
    scoring="accuracy"
)

display("Cross-validation scores:", scores)
display("Mean accuracy:", scores.mean())
display("Standard deviation:", scores.std())

```

```

'Cross-validation scores:'
array([1., 1., 1., 1., 1.])
'Mean accuracy:'
np.float64(1.0)
'Standard deviation:'
np.float64(0.0)

```

✓ Hyperparameter Tuning Using Grid

1. Import the required library

```

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

```

2. Define the parameter grid

```

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

```

- **n_estimators:** *Number of trees.*
- **max_depth:** *Maximum depth of each tree.*
- **min_samples_split:** *Minimum samples required to split a node.*
- **min_samples_leaf:** *Minimum samples required in a leaf node.*

3. Create the GridSearch model

```

rf = RandomForestClassifier(random_state=42)

```

```

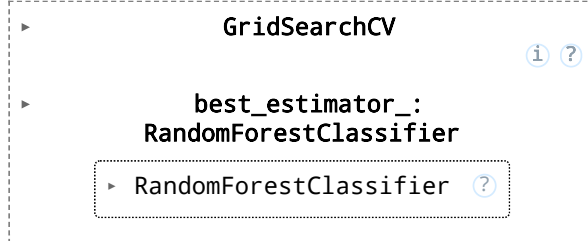
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring="f1_weighted",
    n_jobs=-1,
    verbose=1
)

```

4. Train the model

```
grid_search.fit(X_train, y_train)
```

Fitting 5 folds for each of 36 candidates, totalling 180 fits



5. Find the best parameters

```

display("Best Parameters:")
display(grid_search.best_params_)

display("Best Cross-Validation Score:")
display(grid_search.best_score_)

```

```

'Best Parameters:'
{'max_depth': None,
 'min_samples_leaf': 1,
 'min_samples_split': 2,
 'n_estimators': 50}
'Best Cross-Validation Score:'
np.float64(1.0)

```

6. Evaluate the tuned model

```

best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test)

from sklearn.metrics import classification_report

print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

```

precision	recall	f1-score	support
-----------	--------	----------	---------

0	1.00	1.00	1.00	66
1	1.00	1.00	1.00	163
2	1.00	1.00	1.00	41
accuracy			1.00	270
macro avg	1.00	1.00	1.00	270
weighted avg	1.00	1.00	1.00	270

7. Compare the original and tuned models

```
from sklearn.metrics import accuracy_score

original_accuracy = accuracy_score(
    y_test,
    y_pred_rf
)

tuned_accuracy = accuracy_score(
    y_test,
    best_model.predict(X_test)
)

display("Original Accuracy:", original_accuracy)
display("Tuned Accuracy:", tuned_accuracy)
```